Phase 1 Technical Design – Foundation & Project Setup Appointment Booking SaaS Platform

Document Status: Approved & Frozen Version: 1.0 Phase: Phase 1 Only Related Document: Product & Architecture Requirements v1.0

1. Executive Summary

This document defines the complete technical design for:

Phase 1 – Foundation & Project Setup

The purpose of Phase 1 is to establish a stable, maintainable, and scalable foundation before implementing any business functionality.

Phase 1 focuses exclusively on:

Repository setup Development environment setup Backend setup Mobile setup Database setup ORM initialization Docker setup Environment configuration Documentation Testing foundation

Phase 1 intentionally excludes all business functionality.

No product features should exist after Phase 1 is completed.

1. Purpose

The goal of Phase 1 is to create the technical foundation required for future development.

The foundation should:

Support the approved architecture Be easy to run locally Be easy to maintain Be easy to extend Minimize developer onboarding time Avoid future restructuring

Phase 1 exists to reduce future implementation risk.

1. Scope Included

Phase 1 includes:

Repository Setup Monorepo creation Workspace configuration Package management configuration Backend Setup NestJS application TypeScript configuration Environment configuration Health check Testing setup Mobile Setup Expo application React Native setup TypeScript setup Placeholder application Database Setup PostgreSQL Local development database ORM initialization Infrastructure Docker Compose Local development services Documentation README Local setup guide Development workflow guide Quality Basic tests Type checking Linting configuration Excluded

Phase 1 must not include:

Authentication Users Tenants Memberships Roles Permissions Services Staff Availability Booking engine Branding Notifications Admin functionality

All excluded functionality belongs to future phases.

1. Technical Objectives

Phase 1 should establish:

Objective 1

A working monorepo.

Objective 2

A working backend application.

Objective 3

A working mobile application.

Objective 4

A working PostgreSQL database.

Objective 5

ORM initialization and database connectivity readiness.

Objective 6

A reproducible local development environment.

Objective 7

Clear project documentation.

Objective 8

Basic testing and validation infrastructure.

1. Approved Technology Stack

The approved architecture currently uses:

Mobile React Native Expo TypeScript Backend NestJS TypeScript Database PostgreSQL ORM

Phase 1 decision:

ORM initialization only.

The final ORM selection remains open.

Candidates include:

Prisma TypeORM

The architecture does not depend on either choice.

The ORM decision may be finalized later.

Package Management

Recommended:

pnpm Containerization Docker Docker Compose 6. Monorepo Strategy Objective

Use a single repository.

The repository should contain:

Backend application Mobile application Shared package Benefits Consistency

One repository.

One dependency strategy.

One onboarding process.

Shared Tooling

Shared:

TypeScript Linting Formatting Development scripts Simplicity

Avoids managing multiple repositories.

1. Project Structure

Recommended structure:

appointment-saas/

apps/ api/ mobile/

packages/ shared/

README.md docker-compose.yml package.json pnpm-workspace.yaml apps/api

Backend application.

Responsibilities:

API runtime Future business modules Database integration Security infrastructure

Phase 1 only establishes the application shell.

apps/mobile

Mobile application.

Responsibilities:

User interface Future tenant experience Future booking experience

Phase 1 only establishes the application shell.

packages/shared

Shared package.

Responsibilities:

Shared definitions Shared constants

Not responsible for:

Business logic Security logic Authorization logic 8. Backend Setup Purpose

Establish a running backend service.

Phase 1 Responsibilities

The backend must:

Start successfully Load configuration Expose health status Support testing Support ORM

initialization Health Validation

The backend should provide a simple health check.

Purpose:

Verify application startup Verify deployment readiness Verify development environment Future Readiness

The backend should be structured to support future modules without implementing them.

No future domain functionality should be created.

1. Mobile Setup Purpose

Establish a working mobile application.

Requirements

The application must:

Start successfully Compile successfully Display a placeholder screen Support future navigation Placeholder UI

Example:

Appointment SaaS

Foundation Setup Complete

No additional product functionality is required.

1. Database Setup Database

PostgreSQL.

Purpose

Provide a local development database.

Requirements

The database must:

Start locally Be accessible from the backend Support ORM initialization Not Included

No business entities.

No users.

No tenants.

No appointments.

No services.

No domain tables.

Those belong to later phases.

1. ORM Initialization Purpose

Prepare the application for future persistence.

Phase 1 Responsibilities ORM configuration Connection configuration Client generation if applicable Migration readiness if applicable Non-Goals

Do not create:

User model Tenant model Appointment model

ORM readiness only.

1. Docker Strategy Purpose

Create a reproducible local environment.

Required Service PostgreSQL

The database should run in Docker.

Development Model

Recommended:

Database -> Docker

Backend -> Host

Mobile -> Host

Benefits:

Fast development Easy debugging Simple setup Avoid

For MVP:

Kubernetes Dockerized mobile builds Complex orchestration 13. Environment Configuration Goals

Configuration should be:

Explicit Documented Reproducible Environment Files

Use example environment files.

Real secrets should never be committed.

Backend Configuration

Examples:

Environment name Port Database URL Mobile Configuration

Examples:

API URL Environment name Validation

Applications should fail clearly when required configuration is missing.

1. Shared Package Strategy Purpose

Avoid duplication.

Allowed Shared constants Shared definitions Shared utility types Not Allowed Business

logic Authorization logic Booking logic

The shared package must remain small.

1. Development Workflow

A developer should be able to:

Step 1

Clone repository.

Step 2

Install dependencies.

Step 3

Start PostgreSQL.

Step 4

Configure environment variables.

Step 5

Start backend.

Step 6

Start mobile application.

Step 7

Run tests.

1. Testing Strategy

Phase 1 testing should remain minimal.

Backend Tests

Verify:

Application starts Health check works Configuration loads Mobile Tests

Verify:

Application renders TypeScript compiles Infrastructure Validation

Verify:

Database starts Backend connects Environment configuration works 17. Documentation Requirements

README must include:

Project overview Architecture summary Prerequisites Setup instructions Startup instructions Environment setup Testing instructions Phase 1 scope README Must Also State

Not implemented:

Authentication Tenants Memberships Booking Notifications 18. Acceptance Criteria

Phase 1 is complete when:

Monorepo exists Backend starts Mobile starts PostgreSQL runs ORM initialization works Environment configuration works Tests pass Documentation exists

And:

No business functionality has been implemented.

1. Risks Risk: Future Features Added Too Early

Mitigation:

Strictly limit implementation to foundation concerns.

Risk: Foundation Becomes Difficult To Extend

Mitigation:

Follow modular monolith direction.

Avoid premature modules.

Risk: Environment Drift

Mitigation:

Document configuration.

Maintain environment templates.

Risk: Local Development Inconsistency

Mitigation:

Use Dockerized PostgreSQL.

Document setup clearly.

Risk: Shared Package Growth

Mitigation:

Keep the shared package intentionally small.

1. Review Checklist

Verify:

Monorepo Structure exists Package management works Backend Starts successfully Health check works Mobile Starts successfully Placeholder screen renders Database PostgreSQL runs ORM initialization works Documentation README exists Setup instructions complete Scope

No future functionality exists.

1. Cursor Implementation Instructions

Before implementation:

Explain Planned Work

Cursor must describe:

Files created Files modified Assumptions Scope boundaries Implement Only Phase 1

Cursor must not implement:

Authentication Tenants Memberships Roles Services Staff Appointments Notifications Validate

Run:

Build validation Startup validation Test validation Summarize

Provide:

Files created Files modified Validation results Stop

Do not continue automatically.

1. Final Approval Checklist

Phase 1 is approved only if:

Infrastructure Complete Documentation Complete Tests Passing Scope Preserved Future Features Not implemented 23. Stop Condition

Phase 1 ends immediately after:

Setup completed Validation completed Documentation completed Review completed

No future phase work may begin automatically.

Explicit approval is required before Phase 2 discussion, design, or implementation.

Final Statement

This document defines the complete approved scope of:

Phase 1 – Foundation & Project Setup

It intentionally excludes all business functionality and serves as the implementation baseline for the first phase of development.

End of Document.